Improved SAT-based ATPG on ABC: Incremental PBO with better compaction

113-2 VLSI Testing Final Project

GitHub Repository

Cheng-En Tsai
Dept. of Electrical Engineering
B10901098
b10901098@ntu.edu.tw

Chen-Ching Nieh
Dept. of Electrical Engineering
B10901052
b10901052@ntu.edu.tw

Chen-Yu Hsu
Dept. of Electrical Engineering
B10502139
b10502139@ntu.edu.tw

***Abstract*—As circuit designs become more complex, testing costs rise sharply, making minimal-length Automatic Test Pattern Generation (ATPG) more critical than ever. To address this, we propose an incremental SAT-based approach using Pseudo Boolean Optimization (PBO), in contrast to traditional PODEM-based methods. Our framework encodes the full fault list into a single SAT problem, reducing runtime and improving pattern generation efficiency. It also incorporates a Boolean objective function derived from the circuit structure, along with targeted constraints for the PBO solver. This not only minimizes cost but also enhances test compaction by guiding more selective pattern generation. Moreover, our framework can be naturally extended to support fault diagnosis, reusing the same encoding infrastructure to identify suspected faults from fail logs. Although our current experimental results are limited due to runtime constraints, we outline potential solutions to these challenges and demonstrate the promise of our framework for more efficient ATPG in future work.**

I. Introduction

Automatic Test Pattern Generation (ATPG) is a critical process in digital circuit verification, ensuring the detection of manufacturing defects and improving overall yield. Among the various fault models, the stuck-at fault model remains the industry standard for evaluating the effectiveness of ATPG algorithms, particularly on benchmarks such as the ISCAS85 suite. Traditional ATPG methods often face a trade-off between the number of generated test patterns and the achieved fault coverage, especially as circuit complexity increases.

Recent advancements in Boolean satisfiability (SAT)-based techniques have significantly enhanced the efficiency and scalability of ATPG frameworks. Incremental SAT solving, in particular, allows for the reuse of learned information across multiple related queries, thus reducing redundant computation. Meanwhile, pseudo-Boolean optimization (PBO) provides a systematic approach for minimizing cost functions, such as the number of test patterns, by formulating constraints in a more expressive manner than standard SAT.

In this project, we propose an ATPG framework that integrates incremental SAT solving with pseudo-Boolean optimization, targeting ISCAS85 benchmark circuits represented in the And-Inverter Graph (AIG) format. By leveraging the Berkeley ABC tool for circuit manipulation and fault modeling, our approach aims to systematically generate the minimum set of test patterns required to achieve the highest possible fault coverage for single stuck-at faults.

Beyond pattern generation, our framework is also naturally extended to support fault diagnosis, using the same underlying SAT and PBO infrastructure. Given a set of fail logs, the system can identify and rank suspected faults with high accuracy, further demonstrating the flexibility and robustness of our approach.

Unlike existing methods that prioritize runtime or computational efficiency, our primary objective is to optimize the trade-off between pattern count and fault coverage. Specifically, we focus on demonstrating that a carefully crafted combination of incremental SAT and PBO can produce highly compact test sets without sacrificing fault detection quality, even if it comes at the cost of increased runtime. This study provides both a practical evaluation and a methodological foundation for future research on test pattern minimization in ATPG workflows.

II. Preliminaries

A. Boolean Satisfiability (SAT) Formulation

Boolean Satisfiability (SAT) is a fundamental computational problem that asks whether there exists an assignment to a set of Boolean variables that satisfies a given Boolean formula, typically expressed in Conjunctive Normal Form (CNF) . In the context of ATPG, the behavior of a circuit, along with the presence of faults, is encoded as a SAT problem. Modern SAT solvers are highly effective at exploring the solution space using advanced techniques such as conflict-driven clause learning and efficient decision heuristics, making SAT-based ATPG a robust alternative to classical structural approaches. By formulating ATPG as a SAT problem, test patterns can be generated by finding assignments that distinguish between fault-free and faulty behaviors in the circuit .

B. Pseudo Boolean Optimization (PBO)

Pseudo Boolean Optimization (PBO) extends the SAT problem by incorporating linear constraints over Boolean variables

and introducing an objective function to minimize or maximize . Specifically, PBO solvers seek an assignment that satisfies all Boolean constraints while optimizing a given linear objective, such as minimizing the number of test patterns or care bits. In the context of ATPG, PBO enables the direct encoding of optimization goals—such as test set compaction—within the solving process. This provides a principled way to generate more compact test sets compared to conventional SAT-based ATPG, which typically treats each target independently and does not optimize the overall solution.

III. Related Work

SAT-based ATPG has gained significant attention due to its ability to handle complex fault models and efficiently identify test patterns for hard-to-detect faults. Traditional structural ATPG methods like PODEM are limited in scalability and coverage when applied to modern, highly integrated circuits.

In [1], the authors introduce an efficient SAT-based ATPG method capable of handling all combinations of multiple stuck-at faults. They avoid explicit fault enumeration by using an implicit SAT formulation that incrementally adds constraints without removing any. This "pure" incremental solving approach retains learned clauses throughout the process, which significantly enhances efficiency when tackling extremely large fault spaces. Their method successfully generates complete test vectors for ISCAS89 benchmarks, demonstrating that even with exponential fault combinations, relatively few additional test patterns are needed beyond those for single stuck-at faults.

In a related development, [2] addresses the known challenge of limited test compaction in SAT-based ATPG. By encoding fault detection constraints into the SAT instance and applying formal optimization, they significantly improve test set compactness. Their dynamic compaction strategy uses the implicative strength of SAT solvers to produce minimal test sets, often outperforming previous methods on standard benchmarks.

Our work builds upon these foundations by integrating Pseudo Boolean Optimization (PBO) into an incremental SAT-based ATPG framework. Unlike [1], which focuses on exhaustive fault detection, we target both runtime efficiency and test compaction. Our approach introduces a Boolean objective function derived from the circuit structure, allowing the solver to prioritize pattern selection that minimizes overall test length. Compared to [2], our framework shifts from static SAT encoding to an incremental solving strategy, aiming to maintain learned constraints while adaptively refining pattern selection through PBO constraints.

Together, these prior works motivate our approach, which seeks to combine the scalability of implicit SAT-based fault enumeration with the optimization benefits of PBO for compact and efficient ATPG.

IV. Problem Formulation

A. Problem Description

Given an ISCAS85 benchmark circuit represented as an And-Inverter Graph (AIG), the objective is to generate a minimal set of test patterns that achieves the highest possible fault coverage for single stuck-at faults. The ATPG problem is formulated as a series of Boolean constraints encoding the fault-free and faulty behaviors of the circuit. An incremental SAT approach is used to efficiently identify input vectors that distinguish between fault-free and faulty circuits. To further minimize the total number of generated test patterns, a pseudo-Boolean optimization (PBO) framework is integrated into the SAT-based ATPG process. The problem thus becomes finding the smallest set of input vectors such that all detectable stuck-at faults are covered, subject to the constraints imposed by the circuit structure and fault models.

B. Program Requirements

- **Operating Environment**: The environment should be able to compile Berkeley abc and given PBO solver with *cmake* and *make*. For us, we run our program on Ubuntu 22.04.
- **Time Constraints**: Each circuits we set the time out of 3 hours.
- **Open source tool**:
  - Berkeley abc: https://github.com/berkeley-abc/abc
  - Mixed-bag & PB-OLL-RS: https://bitbucket.org/coreo-group/pbcomp24-cg/src/main/

V. Methodology

This section provides background knowledge and the methodology of our works. Subsection V-A demonstrates how we formulate an incremental SAT formula. Subsection V-B introduces how we concern the PBO constraints. Subsection V-C focuses on the combination of the previous 2 parts.

A. Incremental SAT-based ATPG

1) Fault Modeling: To enable the detection of stuck-at faults using Incremental SAT-based ATPG, the circuit is first transformed so that each potential fault location (e.g., the output of each gate in the AIG) can be selectively controlled during fault simulation. Following the approach described in [1], we introduce two new primary input (PI) variables, x_i and y_i ,for each fault site. These auxiliary variables encode the presence and type of fault as follows:

- $x_i = 0,\ y_i = 1$ indicates a stuck-at-0 fault at site i.
- $x_i = 1,\ y_i = 0$ indicates a stuck-at-1 fault at site i.
- $x_i = 0,\ y_i = 0$ encodes the fault-free (normal) condition at site i.

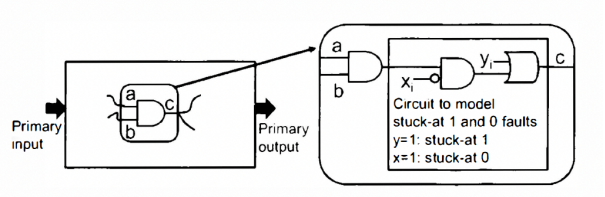


Fig. 1. Modeling stuck-at faults at the output of a gate[1]

This modeling allows the ATPG process to selectively activate and analyze the effect of faults by assigning different values to the variables x_i, y_iin the SAT formulation.

2) Boolean Formula Construction: The behavior of the faulty circuit is described by an augmented logic function $Faulty(in, xy)$ where in is the primary input vector and xy denotes the collection of all variables x_i, y_i, as introduced in [1]. The circuit's output under fault-free and faulty conditions is denoted as $NoFault(in)$ and $Faulty(in, xy)$, respectively. The core SAT formula for ATPG is then:

$$\exists in, xy. Faulty(in, xy) \neq NoFault(in) \tag{1}$$

This formula identifies input vectors in that can detect a specific fault encoded by the assignment to xy.

To avoid redundant solutions and progressively eliminate already detected faults, we adopt an incremental SAT approach. For each newly generated test pattern in_j, a constraint is added to the SAT instance to prevent the solver from rediscovering previously detected faults:

$$Faulty(in_j, xy) = NoFault(in_j) \tag{2}$$

These constraints accumulate over successive iterations, gradually pruning the fault space. Crucially, since the SAT instance is extended without removing prior constraints, the solver retains learned clauses, allowing for efficient exploration across iterations.

Furthermore, while the fault encoding introduced in Section V-A supports multiple stuck-at faults, we constrain the ATPG process to target single stuck-at faults. This is achieved by limiting the number of active faults in the system with the following constraint:

$$\sum_i x_i + \sum_i y_i = 1 \tag{3}$$

This ensures that only one fault (either stuck-at-0 or stuck-at-1) is modeled at a time.

Through this construction, our framework implicitly encodes the complete fault space, while enabling systematic and efficient test pattern generation using incremental SAT solving.

B. Pseudo Boolean Optimization for better compaction

To further improve the compaction of test patterns in the ATPG flow, we incorporate Pseudo Boolean Optimization (PBO) into the SAT-based ATPG framework[2]. The main purpose of adding PBO is to minimize the number of test patterns required for complete fault coverage by maximizing the number of faults detected by each test pattern.

1) Additional Primary Inputs and New Variables: Following the formulation in the SAT-based ATPG stage, each fault site is already associated with auxiliary primary inputs (x_i, y_i) to control stuck-at-0 and stuck-at-1 fault activation. In the PBO-augmented formulation, we introduce further PI variables and local fault detection variables for each fault site. Specifically, for each fault at circuit node a, we add new PIs (e.g.,a_{f0},a_{f1}), which are used to model the activation and propagation conditions for both stuck-at-0 and stuck-at-1 faults.

These local detection variables are then used to represent necessary conditions for a fault to be detected, including:

- **Fault activation**: The node value must be opposite to the fault type (e.g., for s-a-0, node must be 1).
- **Propagation**: All other inputs of the fanout gate must be non-controlling.
- **Path existence**: There must exist a path to an observation point for the fault effect.

2) Constraint Encoding: To encode these conditions in the Boolean formula, we define for each fault location:

- a_{f0}: Indicates whether the activation and propagation conditions for stuck-at-0 at node a are met.
- a_{f1}: Indicates the same for stuck-at-1.

The following types of constraints are added:

- Activation constraint: Ensures a fault can be activated if the corresponding PI values match the required conditions.
$$\begin{cases} a = 1 \rightarrow a_{f1} = 0 \\ a = 0 \rightarrow a_{f0} = 0 \end{cases} \tag{4}$$
- Gate constraint: Propagates the fault only if all other gate inputs have non-controlling values.
Let y be the other input to the gate with controlling value cv:
$$\begin{cases} y = cv \rightarrow a_{f1} = 0 \\ y = cv \rightarrow a_{f0} = 0 \end{cases} \tag{5}$$
- Path constraint: Maintains the propagation to observable outputs through a chain of local detection variables.
Let z be the output of the gate.
$$\begin{cases} z_{f0} = 0 \wedge z_{f1} = 0 \rightarrow a_{f1} = 0 \\ z_{f0} = 0 \wedge z_{f1} = 0 \rightarrow a_{f0} = 0 \end{cases} \tag{6}$$

All these constraints are encoded into the CNF formula and translated into linear inequalities for the PBO solver.

3) Objective Function: The optimization goal in the PBO solver is to maximize the number of faults detected (or equivalently, minimize the negative sum of local detection variables not satisfied). The objective function can be expressed as:

$$F = -\sum_i (a_{i,f0} + a_{if1}) \tag{7}$$

where the sum runs over all undetected faults. By minimizing F, the solver is guided to find an input assignment that

activates and propagates as many faults as possible in a single pattern.

C. Incremental PBO for Improved ATPG

Building upon the incremental SAT-based ATPG framework described in Section V-A and the PBO-based compaction strategy introduced in Section V-B, we propose an **incremental PBO-based ATPG** approach. This method integrates the runtime efficiency achieved in [1] through incremental SAT solving with the improved test set compaction demonstrated in [2] via optimization, the psuedo code of our framework is in Alg 1.

Our goal is to simultaneously generate complete test patterns for the entire fault list while minimizing test length, leveraging the advantages of both techniques.

To accomplish this, the SAT-based fault detection condition (as shown in Equation 1) is combined with local fault activation and propagation constraints (Equations 4–6), and formulated alongside the PBO objective function (Equation 7) to guide test pattern selection for better compaction.

These combined constraints are first expressed in CNF and then converted to the OPB (Optimization Pseudo-Boolean) format required by the PBO solver. The resulting problem instance encodes both detection capability and compaction goals.

For solving, we adopt the Mixed-Bag PBO solver, which ranked first in the 2024 PBO competition. The solver produces the first optimized test pattern, denoted as in_1. After each solution, a **fault dropping constraint**—similar to Equation 8—is added to the OPB formulation to exclude already-detected faults in subsequent iterations.

This process is repeated iteratively: the PBO solver is invoked, a new test pattern is generated, and constraints are updated. When the solver returns `UNSAT`, it indicates that no further test patterns exist that can detect any remaining undetected faults. At this point, all detectable faults have been covered.

This incremental PBO approach tries to enable efficient exploration of the entire fault space, while systematically generating a highly compact and minimized test set. By integrating incremental SAT solving with pseudo-Boolean optimization, our framework tries to achieve comprehensive fault coverage with a reduced number of patterns.

Algorithm 1 Incremental PBO-base ATPG

```
Input: Circuit file, Fault list
Output: Test Patterns
for each fault in Fault list do
    if fault is SA0 then
        Add new PI x_fault
        Insert an AND gate to the fault node with ∼ x_fault
        Insert the corresponding local detection constraint for the fault
    end if
    if fault is SA1 then
        Add new PI y_fault
        Insert an OR gate to the fault node with y_fault
        Insert the corresponding local detection constraint for the fault
    end if
end for
Combine the Faulty and Good circuit using XOR
Transform the circuit into AIG then CNF
Transform into OPB format
Set the objective function
while satisfiable do
    PBO solve the pseudo boolean problem
    PBO generate a single test pattern
    Store the test pattern
    Generate fault dropping constraint with the test pattern
    Concat the CNF with fault dropping CNF
end while
Return the list of generated test patterns
```

VI. Implementation

This section provides an explanation of the program's functionality. Subsection VI-A presents the overall framework and workflow. Subsection VI-B describes the process of fault list generation. Subsection VI-C explains the insertion of fault activation variables (x, y). Subsection VI-D introduces the implementation of PBO constraint gates. Subsection VI-E details the network combination for fault detection. Subsection VI-F covers the construction of constraint circuits for fault dropping. Subsection VI-G discusses AIG optimization strategies. Finally, Subsection VI-H outlines the iterative test pattern generation and feedback process.

A. Overview of the Proposed Framework

To support incremental SAT- and PBO-based ATPG, we extensively customized the Berkeley ABC platform. Our enhancements include new internal data structures and ABC commands for fault list management, fault simulation, constraint generation, and circuit merging. Figure **??** illustrates the overall flow of our proposed framework. These additions transform ABC from a traditional synthesis tool into a comprehensive platform for advanced ATPG algorithm research.

The framework is fully integrated into ABC, enabling the entire ATPG process—from circuit input to compact test pattern output—to be executed entirely within the tool, without external scripting. We also extended ABC to support stuck-at fault modeling and automatic fault list construction, while directly interfacing with external SAT and PBO solvers.

By leveraging ABC's robust synthesis and optimization infrastructure, our framework achieves both runtime efficiency and accessibility. The result is a powerful and extensible environment for evaluating SAT-based ATPG with advanced PBO-based compaction.

B. Fault List Generation

In our ATPG flow, generating a complete fault list can be computationally expensive for large circuits. To reduce

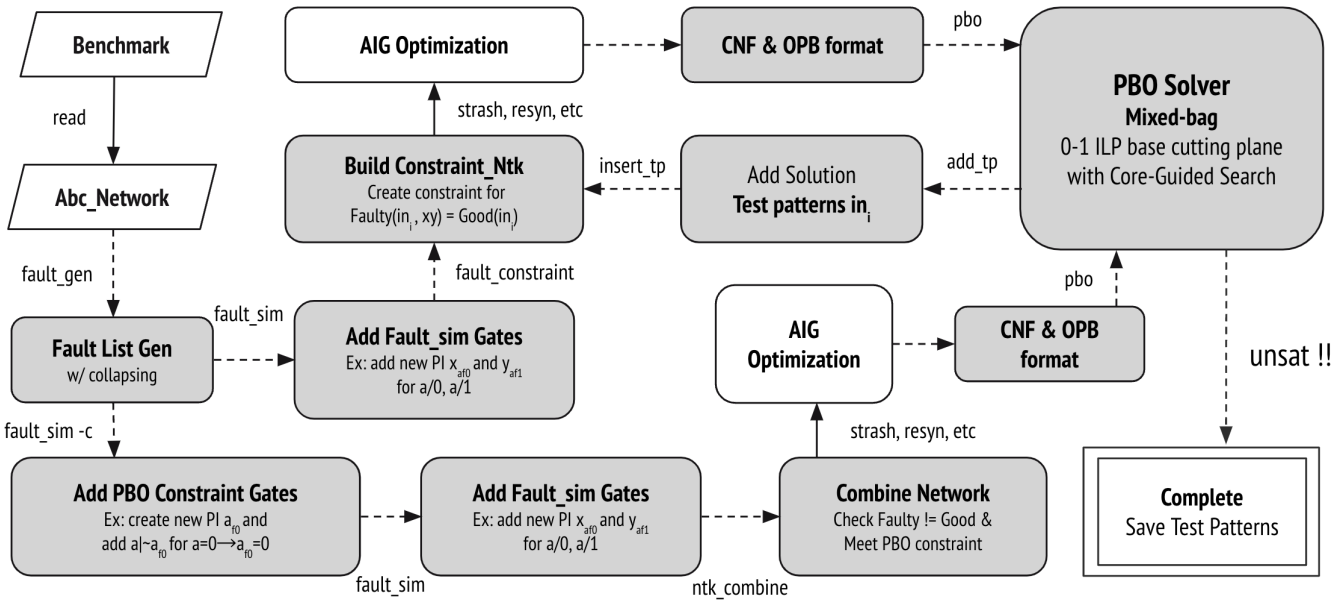


Fig. 2. The Flow Chart of our proposed Incremental PBO-based ATPG on ABC. (Dashed arrows and gray blocks represent our custom ABC commands)

complexity, we apply equivalence fault collapsing (EFC) and the checkpoint theorem to keep only essential faults.

In our ABC-based implementation, the circuit netlist distinguishes between stem and branch nodes:

- For a stem (fanout source), we add a GO (Gate Output) fault.
- For a branch (fanout segment leading to a specific gate input), we add a GI (Gate Input) fault.

By combining EFC, checkpoint reduction, and accurate classification of stems and branches, we ensure an efficient, compact fault list without loss of fault coverage.

We implemented a command named $fault_gen$ to automate the generation and maintenance of fault lists within the ABC environment.

- Basic functionality: For a given AIG network, $fault_gen$ enumerates all potential single stuck-at fault sites, typically at the outputs of AND gates, and constructs a comprehensive fault list.
- Checkpoint support (-c): This option enables incremental or checkpointed fault list generation, allowing the ATPG flow to resume from a saved intermediate state for very large circuits, reducing recomputation overhead.
- Equivalence Fault Collapsing (-l): With this flag, the fault equivalence and dominance relations are used to collapse the fault list, drastically reducing redundant targets, which is crucial to efficiently handling industrial-scale designs.

To assist in flow verification and debugging, we added the $print_faults$ command. This utility allows users to visualize or export the current fault list, including checkpoint status, equivalence classes, and any collapsed information. Such transparency is vital for tracing ATPG progress and validating the correct handling of fault lists at every stage.

C. Insertion of Fault Activation Variables(x, y)

To support the SAT-based and PBO-augmented ATPG algorithms, the $fault_sim$ command was developed. This command automates the insertion of fault activation logic as described in the literature on SAT-based ATPG for stuck-at faults:

- For each fault in the generated list, two auxiliary primary input variables, x_i and y_i, are added.
- The command automatically traverses the AIG, inserting dedicated fault gates at each fault location.
- The resulting logic structure allows the SAT or PBO solver to activate or deactivate faults via PI assignment, matching the desired ATPG strategy.

D. Insertion of PBO Constraint Gates

To implement the local fault detection constraints (Equations 4–6), we first rewrite each implication $p \rightarrow q$ as an equivalent Boolean expression: $\neg p \vee q$. For every fault in the fault list, we create a new primary input (PI) to represent its local detection variable (e.g.a_{f0},a_{f1})

For each constraint, we construct the corresponding logic gate network in the AIG:

- For each implication, an OR gate is created with inputs $\neg p$ and q
- This process is repeated by iterating over all faults and all their respective constraints (activation, gate, and path).

Additionally, for each constraint, we add a new primary output (PO) node driven by the output of the corresponding OR gate. During the SAT or PBO solving phase, we set these PO nodes to logic 1, thereby enforcing the satisfaction of the original implication constraint. This approach ensures that all local detection conditions are encoded directly in the network and

can be systematically checked and optimized by the SAT/PBO solver.

The $fault_sim$ command is further enhanced with a $-c$ option to insert additional logic for PBO constraint gates, thus embedding PBO optimization logic directly into the network.

E. Network Combination for Fault Detection Constraint

To construct the fault detection constraint described in Equation 1, we implement a custom command `ntk_combine` that builds a combined network consisting of both the fault-free and faulty versions of the circuit. This network includes the fault injection logic as well as gates needed to support PBO constraints.

For primary inputs (PIs), the tool merges the PIs from the fault-free and faulty circuits. The additional auxiliary inputs (x_i, y_i) required for fault modeling remain exclusively in the faulty circuit and are preserved for use in the constraint encoding.

For primary outputs (POs), each corresponding pair of outputs from the fault-free and faulty circuits is connected to an XOR gate. These XOR outputs are then combined via a single OR gate to form a new unified PO that represents fault detection—i.e., the output is high if and only if at least one faulty PO differs from its fault-free counterpart.

Additionally, POs that exist only in the faulty circuit (typically due to constraint enforcement or PBO encoding) are preserved separately to allow the PBO solver to evaluate and satisfy the necessary conditions during optimization.

This structural combination enables the generation of a CNF formula that reflects both the behavioral difference between fault-free and faulty circuits and the optimization objectives of the ATPG task.

F. Constraint Circuit for Fault Dropping Constraints

To construct the fault dropping constraints described in Equation 8, we implement the command `fault_constraint`, which generates a constraint network. This network includes both the fault-free and faulty versions of the circuit, but omits the PBO-specific gates used for optimization. The constraint circuit is appended to the original ABC network as a separate module to preserve the integrity of the main ATPG flow. It is used solely for constraint generation and does not interfere with the test pattern search structure.

The network construction process mirrors that of `ntk_combine`. The primary inputs (PIs) of the constraint network are shared between the fault-free and faulty circuit models. Auxiliary inputs (x_i, y_i) are retained for fault modeling, as before. For primary outputs (POs), each corresponding pair of outputs from the two circuits is connected through an XOR gate, and the resulting XOR outputs are combined using a NOR gate. This final output evaluates to logic high only when all outputs match—indicating no detectable fault—thus enabling the detection of already-covered faults.

To dynamically incorporate test patterns during ATPG, we introduce the commands `add_tp <test_pattern>` and `insert_tp`. The `add_tp` command stores each test pattern generated by the PBO solver, while `insert_tp` maps the constant logic values (0 or 1) of the latest test pattern onto the corresponding PIs in the constraint network. This leaves only the fault control variables (x_i, y_i) unconstrained.

The resulting constraint network, now partially instantiated with fixed input values, can be translated into CNF and appended to the SAT or PBO instance. This enforces the condition that previously detected faults are excluded from future solutions, enabling iterative fault dropping and efficient complete fault list coverage.

G. AIG Optimization

With the complete circuit structure in place, the next step is to transform the logic network into CNF format for use with SAT or PBO solvers. To reduce clause overhead and improve solver runtime, circuit optimization is essential.

We take advantage of ABC's strength in AIG-based logic optimization by using And-Inverter Graphs (AIGs) as the intermediate representation. AIGs provide a compact and uniform structure, making them ideal for structural transformations and simplification.

The transformation flow begins by converting the circuit into an AIG using ABC's built-in commands `fraig` and `strash`. Once in AIG form, we apply structural optimizations using commands such as `resyn2` and `compress2`, which reduce logic depth and redundancy while preserving functional correctness.

Finally, the optimized AIG is converted into CNF using the `write_cnf` command. This CNF serves as the input to the SAT or PBO solver for ATPG, with a reduced number of clauses and improved overall efficiency.

H. Iterative Test Pattern Generation and Feedback Flow

1) Parse CNF into OPB format: The DIMACS CNF and the OPB format has similar structure, thus can be transformed easily. For example, A clause $x_1 \vee \neg x_2$ can be encoded as $1\ x_1\ 1\ \sim x_2 >= 1$, where $\sim x_2 = 1 - x_2$. In order to incrementally execute the PBO solver, we need to map the variables x and y in the adding clauses to the variables in the main formula.

2) Run PBO solver: In the self-defined command `pbo` in abc, we fork a subprocess to execute the PBO solver **mixed-bag**[3], then receive and parse the solver output to get the pattern.

3) Fault Dropping: After receiving the pattern, we do a fault simulation and update the objective function of the pbo problem to eliminate all detected faults, so that the pbo will not try to detect the detected faults, but focus on maximizing the possibility of detecting the other undetected faults.

VII. Results

During the evaluation of our ATPG framework on the ISCAS85 benchmarks, we observed that scalability becomes

TABLE I
COMPARISON OF PODEM AND OUR APPROACH

	PODEM			Ours			Ours w/ fsim		
ckt	**#TestPatterns**	**FaultCoverage**	**Runtime**	**#TestPatterns**	**FaultCoverage**	**Runtime**	**#TestPatterns**	**FaultCoverage**	**Runtime**
c17	7	**100%**	**0.018s**	6	**100%**	**0.542s**	8	**100%**	**0.418s**
c423	20	**17.88%**	**0.998s**	20	**17.88%**	**760s**	20	**17.88%**	**525s**
c499	**74**	**96.48%**	**0.337s**	**N/A**	**N/A**	**N/A**	**16**	**46.79%**	**7hr**

TABLE II
CNF SIZE FOR BASE AND ADDING FAULT CONSTRAINTS

Circuit	Base CNF	Added CNF
C17	73 vars & 154 clauses	30 vars & 40 clauses
C432	1899 vars & 3981 clauses	821 vars & 1300 clauses
C499	3938 vars & 8591 clauses	1600 vars & 2600 clauses

a significant challenge as circuit size increases. As shown in Table I, the growth in the number of gates and faults leads to a substantial increase in both the number of generated clauses and local detection constraints. The corresponding variable and clause counts are detailed in Table II. This rapid growth results in extremely large CNF and PBO instances, which quickly surpass the memory and computational capabilities of state-of-the-art PBO solvers.

One key observation is that the explosion in clause and constraint counts directly impacts the solver's ability to process the problem. In practice, for small and medium-sized circuits, our method is able to generate compact test sets efficiently. However, for larger benchmarks, the solver often fails to complete, or runs out of memory, due to the overwhelming number of constraints introduced by our fine-grained modeling and exhaustive constraint generation.

An additional factor contributing to this issue is our choice of PBO solver. In order to maximize solution quality, we selected a "mixed-bag" solver that uses an ILP-based cutting plane approach. While such solvers can produce optimal or near-optimal results, they are less robust when faced with extremely large, highly-constrained instances, especially compared to native SAT solvers optimized for sparse or incremental formulations. Our implementation did not fully account for the limitations of ILP-based approaches under these conditions, which further exacerbated the scalability bottleneck.

These results suggest a clear trade-off between solution optimality and scalability in SAT+PBO-based ATPG frameworks. Future work will focus on utilizing SAT-based PBO solver, reducing the number of constraints through smarter encoding, constraint pruning, or hierarchical test generation strategies, as well as exploring solver architectures that can better handle large-scale industrial designs.

VIII. DIAGNOSIS

A. Problem Description

Fault diagnosis is a critical process in Automatic Test Pattern Generation (ATPG), aimed at identifying the specific defects in a circuit that cause observed failures. While ATPG focuses on generating test patterns to detect faults, diagnosis works in reverse—starting from fail logs or test responses, it attempts to pinpoint the actual fault locations within the circuit.

B. Methodology

Using the previously defined fault activation variables (x, y), we construct a CNF formula for each test pattern in the test set. For each pattern, we assign the primary inputs to match the values in the test pattern and set the primary outputs to the expected values—using the failing output values from the fail log, or the good circuit outputs if not specified.

$$Faulty(in_j, xy) = test\ output \tag{8}$$

We then aggregate all individual CNFs into a single combined CNF, where the fault-related variables (x, y) are shared across all patterns. By solving this unified CNF, we can achieve 100% diagnosis accuracy, as the solution must explain all observed failures consistently.

C. Implementation

First, within the same framework, we reuse the existing commands for generating the fault list and inserting fault activation variables (x, y), ensuring consistency and modularity across the diagnosis process.

Next, we developed the `add_po` command to enforce the expected primary output (PO) values. This is done by appending a NOT gate to each PO expected to be 0, effectively converting it to 1, and then combining all PO signals using an AND gate. The output of this AND gate is then constrained to 1, ensuring that all original POs match the expected test output for that pattern.

We then use the built-in `&write_cnf` command in Berkeley-ABC to generate the corresponding CNF file, which is subsequently solved using the Kissat SAT solver.

- Kissat: https://github.com/arminbiere/kissat

D. Result

We achieve 100% diagnosis accuracy on all fail logs of the `c17` circuit, with each case solved in under one second. However, as the circuit size and the number of faults increase, the number of constraint clauses—used to limit the number of suspected faults—grows exponentially. Without those clauses, the solver returns a solution with up to 70 simultaneous suspect faults that match the fail log, which is clearly not the intended result. This exponential growth severely limits scalability, making it challenging for the SAT solver to find meaningful solutions within a practical runtime.

IX. Discussion

A. Run time

The run time of our approach is significantly higher than the traditional ATPG algorithm like PODEM. Take ISCAS-85 C499 circuit as an example, the original PBO problem already has 3938 variables and 8591 clauses, solving it takes about 1 minute. Beyond that, adding every new pbo constraint for fault dropping will increase more than 1600 variables and 2600 clauses to the original problem. The scale of the problem grows so dramatically that the ILP-based solver cannot handle it after only a few iterations. In addition, the solver restarts every time a new problem is generated, so the learned conflicts cannot be inherited by the later problem. In conclusion, the scale of the PBO-base ATPG problems is too large for the state-of-the-art ILP-based PBO solvers.

B. Fault Coverage & Test Pattern Number

Our approach achieves the same fault coverage as the PODEM algorithm, with one fewer test pattern in the c17 circuit and the same number of patterns in the c432 circuit. This result demonstrates that our PBO approach effectively shortens the test length, though at the cost of increased run time.

X. Future Work

A. Incremental SAT

In our approach, the PBO problem is solved by mixed-bag[3], which is an ILP-based solver utilizing cutting planes conflict analysis for PBO problems. However, to fully take advantage of incremental SAT, we need to use a SAT-based PBO solver which will keep all the learned clauses. Since we do not remove clauses in the incremental SAT scenario, keeping the learned clauses will greatly reduce the run-time of the PBO solver.

XI. Job Division

A. Chen-Ching, Nieh, B10901052

- Research on the problem and design the overall framework of the program
- PBO Solver
- PBO Parser
- PBO Commands
- Diagnosis

B. Cheng-En Tsai, B10901098

- Research on the problem and design the overall framework of the program
- Fault Modeling
- Fault Simulation
- Fault Constraint Network
- Test Patterns

C. Chen-Yu, Hsu, B10502139

- Research on the problem and design the overall framework of the program
- Fault List
- PBO Constraints

References

[1] M. Fujita and A. Mishchenko, "Efficient sat-based atpg techniques for all multiple stuck-at faults," in *2014 International Test Conference*, pp. 1–10, 2014.

[2] S. Eggersglüß, R. Wille, and R. Drechsler, "Improved sat-based atpg: More constraints, better compaction," in *2013 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pp. 85–90, 2013.

[3] J. Elffers and J. Nordström, "Divide and conquer: Towards faster pseudo-boolean solving," in *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, IJCAI-18*, pp. 1291–1299, International Joint Conferences on Artificial Intelligence Organization, 7 2018.